gemstone-py Observability

Tracing, metrics, and slow-operation logging

Generated from the Markdown sources in gemstone-py/docs
Assets are local SVG illustrations stored in the repository.

Contents

Observability

`gemstone-py` can emit tracing spans, metrics, and slow-operation log records from the core session API. The default is no-op, so existing applications do not need to configure anything and do not pull in observability dependencies.

Install Optional Adapters

The built-in OpenTelemetry and Prometheus adapters use optional dependencies:

```
python -m pip install "gemstone-py[observability]"
```

You can also pass your own objects that implement the small protocols in `gemstone_py.observability`.

Trace GCI Calls

Pass a tracer when creating a session:

```
from opentelemetry import trace
from gemstone_py import GemStoneConfig, GemStoneSession, OpenTelemetryTracer

tracer = OpenTelemetryTracer(trace.get_tracer("my-app.gemstone"))

with GemStoneSession(config=GemStoneConfig.from_env(), tracer=tracer) as session:
    session.execute("1 + 1")
    session.perform_oop(session.resolve("System"), "myUserProfile")
```

Each observed operation creates a span named like:

```
gemstone.session.execute_oop
gemstone.session.perform_oop
gemstone.session.commit
gemstone.session.abort
```

Common span attributes include:

Attribute	Meaning
`operation`	Session operation name, such as `execute_oop` or `perform_oop`.

Attribute	Meaning
`stone`	Configured GemStone stone name.
`host`	Configured GemStone host.
`status`	`ok` or `error`.
`duration_ms`	Wall-clock duration for the operation.
`selector`	Smalltalk selector for `perform_*` calls.
`argc`	Argument count for `perform_*` calls.
`source_length`	Source string length for eval/execute calls.

The instrumentation deliberately records source length rather than full Smalltalk source, so normal traces do not expose application data or secrets.

Record Metrics

Pass a metrics collector alongside the tracer, or use metrics on their own:

```
from gemstone_py import GemStoneConfig, GemStoneSession, PrometheusMetrics

metrics = PrometheusMetrics()

with GemStoneSession(config=GemStoneConfig.from_env(), metrics=metrics) as session:
    session.execute("1 + 1")
```

The session emits:

Metric	Labels	Meaning
`gemstone_py_session_operations`	`operation`, `status`	Operation counter.
`gemstone_py_session_duration_ms`	`operation`, `status`	Duration samples in milliseconds.

For existing Prometheus setup, pass your service's registry to `PrometheusMetrics(registry=...)`.

Pool And Query Signals

`GemStoneSessionPool`, `GemStoneThreadLocalSessionProvider`, and `AsyncSessionPool` accept the same `metrics=` and `tracer=` objects:

```
from gemstone_py import GemStoneConfig, GemStoneSessionPool, PrometheusMetrics

metrics = PrometheusMetrics()
pool = GemStoneSessionPool(
    maxsize=4,
    config=GemStoneConfig.from_env(),
    metrics=metrics,
)
```

The pool emits:

Metric	Labels	Meaning
`gemstone_py_pool_events`	`event`, `provider`, `provider_type`, optional `reason`	Pool lifecycle/event counter.
`gemstone_py_pool_acquire_wait_ms`	`provider`, `provider_type`	Time spent waiting for a pooled session.

Pool spans are named like:

```
gemstone.pool.session_acquired
gemstone.pool.session_released
gemstone.pool.session_discarded
gemstone.pool.acquire_timeout
```

Chunked query iteration emits session-level spans when tracing is configured on the session:

```
gemstone.session.query_iter_chunk
gemstone.session.query_iter
```

Those spans include the collection name, chunk size, chunk range, number of chunks fetched, and total yielded rows.

Slow Operation Log

Enable structured warning logs for slow operations:

```
import logging
from gemstone_py import GemStoneConfig, GemStoneSession

logging.basicConfig(level=logging.WARNING)

with GemStoneSession(
    config=GemStoneConfig.from_env(),
    slow_query_threshold_ms=100.0,
) as session:
    session.execute("1 + 1")
```

Slow records are written through:

```
gemstone_py.slow_queries
```

The log record includes the operation, duration, stone, host, session id, status, and error type when one is available.

Async Sessions

`AsyncSession` creates and uses the same underlying `GemStoneSession`, so pass the same keyword arguments:

```
from opentelemetry import trace
from gemstone_py import GemStoneConfig, OpenTelemetryTracer
from gemstone_py.aio import AsyncSession

tracer = OpenTelemetryTracer(trace.get_tracer("my-app.gemstone"))

async with AsyncSession.connect(
    config=GemStoneConfig.from_env(),
    tracer=tracer,
    slow_query_threshold_ms=100.0,
) as session:
    await session.execute("1 + 1")
```

Custom Collectors

The protocols are intentionally small:

```
class MyMetrics:
    def increment(self, name, labels=None, value=1):
        ...
```

```
    def record_duration(self, name, labels, duration_ms):
        ...
```

This keeps the core package independent from any one telemetry stack while still making the production hooks explicit.

> This PDF was generated locally from the Markdown sources in `docs/`. If you edit the source files, rerun `python docs/build_pdf_docs.py`.